

Deterministic Cognitive Model (DCM)

Term: Deterministic Cognitive Model

Acronym: DCM

Architecture class: AGI-capable governed cognitive architecture

Reference implementation: CRM-FFE (Crowell Reasoning Model — FFE fork)

Companion documents:

[\[THE_PARADOXICAL_SHIFT_GGUF_AND_GOVERNED_ASSEMBLY.md\]](#) ([THE_PARADOXICAL_SHIFT_GGUF_AND_GOVERNED_ASSEMBLY.md](#)) ·

[\[CRM_COGNITIVE_FLOW_OVERVIEW_FOR_DOCS.md\]](#) ([CRM_COGNITIVE_FLOW_OVERVIEW_FOR_DOCS.md](#))

Author: Ray David Crowell

1. Formal definition

A **Deterministic Cognitive Model (DCM)** is an **integrated model architecture class** in which general cognitive behavior is produced by a **governed, auditable, safety-supervised runtime** that unifies three built-in faculties—neural articulation, governed cognitive steering, and **embedded factual cognition** (native in-process physics, chemistry, mathematics, units, logic, and related domain engines)—resolving intent, map authority, procedural structure, inference control, memory state, subconscious administrative policy, and multi-instance coherence **before and during** neural token generation. The neural component—typically a compressed language-model checkpoint (e.g., GGUF) or, alternatively, a state-space sequence engine—functions as a **controlled language articulator**, not as the sole authority for truth, procedure, safety, or task completion. **Factual domains are part of the model architecture**, not external API services routed around it.

Under identical inputs, policy versions, and resource availability, a DCM is designed to yield **replay-stable** governance, embedded-domain execution, and auditable provenance. Outputs are **evidence-bound** where claims depend on in-process computed facts; fluent language alone is insufficient for release. Quantitative and logical claims are subject to **mathematical safety**: deterministic inline computation within the integrated runtime, bounded inference-control magnitudes, and embedded-domain results rather than unconstrained numerical or symbolic assembly by the neural substrate alone. Capability may be **transferred and refined** through governed teaching artifacts (maps, frameworks, memory programs) and real-time learning on routing and memory state **without** requiring full weight retraining of the neural substrate for each new workflow.

The architecture is **edge-native by design**: it activates in working memory only the maps, shards, embedded truth domains, and control state **relevant to the active turn**, rather than requiring full corpus residency or datacenter API dependency for core factual grounding. A conforming DCM may therefore operate on commodity and constrained edge hardware while preserving governance, embedded truth, and audit contracts.

DCM is proposed as the **evolutionary successor** to token-first large language model (LLM) product architecture: the industry's primary engineering object shifts from optimizing unconstrained generative fluency alone to **dependable, teachable, auditable, safety-administered cognitive infrastructure** with an interchangeable neural articulation layer.

1.1 The tripartite cognitive contract (definitional)

What makes DCM a distinct architecture class—not a larger checkpoint, not a cloud API router, and not a chatbot with external plugins—is a **tripartite cognitive contract**. A DCM is **one integrated cognitive system** with three built-in faculties that must converge **before** user-facing language is released:

Faculty	Role	What it must not be
Neural articulation	How to say it — language surface, phrasing, fluency	The authority for truth, procedure, or safety
Governed cognitive runtime	How to reason and steer — intent, maps, procedures, dynamical state, inference control, memory, supervision	Prompt text, hidden context, or cloud orchestration alone
Embedded factual cognition	What is factually grounded — native physics, chemistry, math, units, logic, and related domain engines built into the architecture , activated and integrated as part of the turn	External API calls, retrieved passages, or post-hoc fact-checking of fluent text

The neural substrate (e.g., GGUF) supplies **words**. The governed runtime supplies **operational cognition and steering**. The **embedded truth substrate** supplies **facts**—deterministic, in-process, domain-native results (physics, chemistry, quantities, logic, and related governed compute) that the architecture treats as authoritative over conflicting token assembly.

This is not API routing. In token-first and agent-orchestration designs, the model guesses, then something external may be called to verify or retrieve. In DCM, **factual domains are structurally part of the model**—woven into the same cognitive pipeline as steering and articulation, running **in-process on the device**, not as a separate cloud service layer. The architecture **deterministically activates** which embedded truth domains apply to the turn, runs them under map authority **before and during** stage synthesis, and **integrates** verified results into evidence state and cognitive stability paths—not as pasted context for the checkpoint to paraphrase, but as the **factual ground** the checkpoint is governed to speak from.

Factual response is not assembled by the checkpoint and then verified. The integrated truth substrate establishes what is true for the turn first; the GGUF articulates language **back through** that already-established factual and procedural state.

Embedded factual cognition is therefore not an accessory or plugin marketplace. It is approximately **one third of the cognitive process** in DCM: co-equal with governed steering and distinct from raw language generation. Remove it and the system collapses to fluent guessing; remove governed steering and embedded truth becomes inert libraries; remove neural articulation and the system cannot speak—but **neither faculty can be replaced by the checkpoint alone**. This tripartite convergence—words, steering, and **built-in factual grounding**—is the structural reason DCM can be dependable, teachable, and auditable on edge hardware while token-first systems require scale to simulate the same surface behavior.

2. Taxonomic placement

Class	Primary object of design	Typical failure mode
LLM	Next-token prediction over a fixed weight checkpoint	Plausible but ungrounded or non-repeatable assembly
Neurosymbolic system	Neural–symbolic representation fusion (various bridges)	Bridge learning cost; inconsistent authority between layers
SSM / state-space sequence model	Recurrent state evolution inside the forward pass	No native tool, map, or governance contract

DCM	<i>Integrated cognitive model: neural articulation + governed steering + embedded factual cognition (physics, chemistry, math, etc.)</i>	<i>Misconfiguration of maps/policy (detectable via audit), not silent softmax drift</i>
Agent / API-orchestrated LLM	<i>Checkpoint plus external service calls</i>	<i>Cloud routing; facts fetched or verified after fluency; no native factual substrate</i>
Tool-augmented LLM	<i>Checkpoint plus bolt-on utilities</i>	<i>Reactive plugins; facts quoted in prompts; truth not structurally part of the model</i>
RAG-augmented LLM	<i>Retrieved passages fed into context</i>	<i>Retrieval substitutes for compute; no embedded domain engines or evidence ledger</i>

DCM is **not** a replacement for neural language modeling and **not** a cloud orchestration pattern. It is a **reclassification** of what a cognitive model is: an integrated architecture in which **factual domains are built in**, governed steering coordinates them, and the checkpoint articulates from established ground—not an oracle that occasionally calls out for help.

3. Necessary properties (architecture class)

The following properties are **definitional** for DCM. An implementation may gate features opt-in, but the **architecture contract** must support them.

3.1 Determinism and replay

- Turn routing, fallback chains, clause enforcement, consistency gating, and supervisory actions are **policy-driven** and **replay-stable** under fixed operating conditions.
- Stochasticity in neural sampling (e.g., temperature) is **explicitly scoped** to language articulation and does not override governed routing or evidence contracts.

3.2 Pre-generative resolution

- *Intent, map eligibility, embedded truth-domain activation, execution-contract binding, and procedural framework binding* are resolved prior to unconstrained answer synthesis.
- The architecture **deterministically decides** which built-in truth domains (physics, chemistry, math, units, logic, and related native engines) are admissible for the turn—not through model discretion or external API selection, but through map authority, governed action binding, and in-process execution contracts.
- Only the **relevant embedded truth domains** for the active turn or stage are activated in working memory rather than requiring full domain-corpus residency at once.
- In-process truth execution produces **authoritative evidence** consumed by downstream stages, inference stability paths, and release gates.

3.3 Evidence-bound release

- Response candidates are subject to **obligation coverage** (required clauses), **claim–evidence consistency**, and **repair or abstention** policy before user-facing release where the domain requires it.

3.4 Inference control distinct from prompting

- Behavior is shaped by **mandatory inference-control and adapter-steering mechanisms** (control vectors, depth-localized binding, layer gain profiles) rather than by reliance on prompt text as the primary cognitive channel.
- Prompt steering is **not** the authoritative governance surface.

3.5 Teachability without weight retraining

- Domain and procedural capability are transferable via **governed artifacts** (task maps, frameworks, memory programs, teaching packs) with validate–promote policy.
- Real-time learning updates **routing reliability, memory geometry, and map artifacts** under bounded rules; it does not require full-model fine-tuning for each operational workflow.

3.6 AGI-capable architecture (qualified)

DCM is **AGI-capable** in the **architectural** sense: it supports **cross-domain task execution** under unified governance—intent routing, tool use, memory, teaching, supervision, and audit—rather than in the sense of claiming a fixed parameter count or checkpoint size sufficient for human-level generality. General breadth of language and world coverage may still scale with the neural substrate; **dependability, teachability, and auditability** scale with the DCM runtime.

3.7 Auditability (necessary)

Auditability is **not** an optional product feature in DCM; it is a **structural requirement**.

A DCM implementation must support:

1. **Append-only decision lineage** — governance actions, gate outcomes, route selections, and fallback states persisted with stable schema semantics.
2. **Cross-artifact provenance linkage** — query identity, tool invocations, clause resolution, consistency-gate results, supervisory memory references, and teaching-contract participation linked into a **single reconstructable chain** per turn.
3. **Reasoning traces** — phase-structured records of cognitive execution suitable for operator review and forensic analysis.
4. **Dual-channel audit where training evolution is present** — operator-bounded rollup traces and training-scoped selection audit with full routing forensics, so improvement loops do not require exposing sensitive internals to end users.
5. **Replay validation** — ability to verify that identical inputs and policy state reproduce routing, gate, and governance outcomes (within declared neural-sampling scope).
6. **Promotion gates** — artifact changes (maps, frameworks, memory bundles) admitted to active production only through explicit validate–archive–promote policy with recorded evidence.
7. **Infrastructure vs. reasoning failure classification** — adapter, pipeline, and tool failures distinguished from cognitive miss so benchmarks and compliance reviews are not confounded.

Without auditable provenance, a system may be **tool-augmented** or **workflow-guided**, but it does not meet the DCM class contract.

3.8 Safety administration and subconscious governance (necessary)

DCM treats **safety as an administrative control plane**, not as post-hoc moderation of generated text alone.

A conforming implementation must support:

1. **Subconscious identity governance** — a dedicated supervisory authority with **hard action rights** over release paths: allow, rewrite strategy, restart cycle, request user context, or block/abstain under bounded policy.
2. **Pre- and post-release evaluation** — supervisory checks applied to interpreted intent and assembled response posture against immutable hierarchical value priorities (paternal safety: best for user under best for humanity).
3. **Closed supervisory action set** — governance actions are enumerable and auditable; supervision **does not** invent new tool routes or bypass deterministic execution contracts.
4. **Bounded restart and escalation** — restart and context-request pathways operate under guard counters and deterministic terminal behavior; unbounded retry loops are excluded by architecture.

5. **Independence from reproducibility locks** — safety administration (subconscious governance) is **orthogonal** to STRICT reproducibility/testing locks; neither channel may silently override the other's mission.
6. **Append-only supervisory decision memory** — pre-check, post-check, reason codes, actions taken, and outcome labels persisted for audit and bounded policy telemetry.

Safety in DCM is **operational administration** with provenance, not a separate classifier bolted onto fluent output.

3.9 Interconnected instances (necessary)

DCM admits **multi-connected deployment**: multiple runtime instances (e.g., parallel operator sessions, launcher instances, or distributed presences) that:

1. **Synchronize state** under deterministic sharing contracts where required.
2. **Preserve single-turn authority convergence** — one stable execution contract and one authoritative outcome per user turn, without split-governance drift across instances.
3. **Record instance participation** in audit artifacts where instance-level forensics are required.

Interconnected instances enable commercial topologies (multi-tab operators, multi-site presence, personal agent fleets) without reducing governance to independent chat silos.

3.10 Edge-native resource governance (necessary)

DCM is architected for **constrained and edge deployment**, not datacenter-only residency.

A conforming implementation must support:

1. **Demand-loaded cognitive assets** — memory shards, maps, and control structures resident in accelerator or system memory **only when selected** for the active turn or stage, not as mandatory full-corpus preload.
2. **Tiered memory pyramid** — stratified retention policy (e.g., hot preload, demand load, lazy disk, archive) so resource use scales with **relevance**, not with total knowledge base size.
3. **Interchangeable small neural substrates** — demonstration that governed behavior is not contingent on multi-hundred-billion-parameter checkpoints; articulation quality may scale with substrate size, but **governance, audit, and teachability** are runtime properties.
4. **Phone-class and commodity-hardware trajectory** — architecture contract does not assume continuous datacenter availability for core cognitive operation.

Edge-native loading is a **structural** property: intelligence is engineered to be **present locally**, not streamed as datacenter privilege alone.

3.11 Mathematical safety (necessary)

DCM separates **language assembly** from **quantitative truth** through **mathematical safety** requirements:

1. **Deterministic compute spine** — arithmetic, units, symbolic evaluation, and related quantity operations execute through **inline deterministic engines** with replay-stable semantics where the domain requires it, rather than through unconstrained neural hallucination of numbers.
2. **Tool-authoritative numerics** — when a truth engine or calculator produces a result, that result takes **evidence authority** over conflicting token assembly in governed domains.
3. **Bounded inference-control magnitudes** — adapter and binding injection paths operate under **norm clamps and layer gain profiles** so steering signals cannot diverge without bound on small substrates.
4. **Bounded learning updates** — real-time geometry and reliability updates on memory and routing state operate under **capped deltas and gated adoption** so failed or unsafe turns do not corrupt taught state.
5. **Replay-stable mathematical paths** — identical inputs and policy state reproduce compute and routing outcomes within declared scope, supporting audit and regulatory review of quantitative claims.

Mathematical safety is what makes DCM commercially viable in finance, engineering, medicine-adjacent workflows, and operations: **the system can show the math, not merely sound like it did the math.**

3.12 Embedded factual cognition and integrated truth domains (necessary)

Embedded factual cognition is a **definitional pillar** of DCM—co-equal with governed cognitive steering and neural articulation (see §1.1). Physics, chemistry, mathematics, units, logic, and related domains are **not external services the model calls**. They are **native faculties of the architecture**—deterministic compute and domain engines **integrated into the cognitive model itself**, providing the factual grounded basis on which language is generated.

This is what separates DCM from API-orchestrated agents, tool-augmented chat, RAG wrappers, and cloud-routed “agentic” stacks. Those patterns treat truth as something **fetches or verified around** the model. DCM treats truth as something **built into** the model.

A conforming implementation must support:

1. **In-process embedded truth substrate** — domain engines (physics, chemistry, math, units, logic, materials, finance, code, time, geo, and related governed compute) run **inside the cognitive runtime** on the device, not as mandatory external API hops. The factual layer is structural infrastructure, not a peripheral integration.

2. **Deterministic truth-domain activation** — from interpreted intent and operational actions, the architecture resolves **which embedded domains** are authorized for the turn under truth-map and execution-contract policy. Activation is replay-stable and auditable, not model-discretionary and not cloud-router discretion.
3. **Truth-map authority** — registry-backed truth maps match turn conditions, extract admissible rules, and bound domain scope so factual governance can evolve through teaching without breaking deterministic contracts.
4. **Relevance-scoped domain activation** — only the embedded truth domains **pertinent to the resolved turn** are activated in working memory; the full cross-domain inventory is part of the architecture but not required resident at once.
5. **Unified native truth surface** — cross-domain engines participate as **one integrated factual faculty** under a single authority contract, not as a marketplace of disconnected plugins or third-party APIs.
6. **Pre-generative and stage-scheduled execution** — embedded truth runs **before** (and selectively after) neural stage synthesis so a **factual ground** exists **prior to** language assembly, not as corrections after fluency.
7. **Structural fact integration** — verified results enter **evidence ledgers**, stage binding, consistency gates, and cognitive stability paths. Facts are part of the model's operating state; they are not pasted retrieval snippets for the checkpoint to paraphrase.
8. **Stage-level evidence binding** — each procedural stage consumes only admissible embedded-domain outputs with provenance continuity from execution ledger to structured findings.
9. **Evidence readiness and safe degradation** — when required facts cannot be established under policy, the system follows deterministic fallback, repair, or abstention paths rather than fabricating completion behavior.
10. **Post-assembly consistency** — claim–evidence gates ensure released language remains aligned with embedded-domain results; conflicts trigger repair or abstention, not silent override by the neural substrate.

Why this defines DCM: Token-first systems conflate fluency with truth. API-orchestrated agents bolt **external** verification onto the same conflation. DCM **integrates factual cognition into the model**: the checkpoint handles **words**, the governed runtime handles **procedure and steering**, and the **embedded truth substrate** supplies **physics, chemistry, math, and related grounded fact** that the checkpoint is then governed to articulate. That integration—not cloud routing, not plugin marketplaces—is the revolutionary structural fact: **a cognitive model with a built-in factual foundation, not a language model that sometimes calls for facts.**

4. Architectural layers (factual summary)

A conforming DCM stack typically comprises, at minimum:

Layer	Function
<i>Interpretation</i>	<i>Pre-interpretive conditioning; parallel intent resolution; deterministic intent pruning</i>
<i>Map confidence & authority</i>	<i>Hybrid map scoring; eligibility; conflict arbitration</i>
<i>Embedded truth substrate</i>	<i>Native domain engines (physics, chemistry, math, etc.); truth-map authority; relevance-scoped activation</i>
<i>Execution contract</i>	<i>Governed domain binding; fallback policy; unified in-process truth surface</i>
<i>Procedure routing</i>	<i>Framework and persona selection; multi-stage procedural execution</i>
<i>Factual execution</i>	<i>Pre- and post-stage embedded-domain runs; meaningfulness qualification; evidence ledger; structural fact integration</i>
<i>Dynamical cognition</i>	<i>Pre-token state evolution; modality competition</i>
<i>Inference control</i>	<i>Control-vector construction; layer-shaped adapter injection; optional depth-localized association binding</i>
<i>Obligation & consistency</i>	<i>Clause coverage; claim–evidence gating</i>

Mathematical safety	<i>Inline deterministic compute; tool-authoritative numerics; bounded control and learning</i>
Subconscious administration	<i>Pre/post paternal governance; closed action set; decision memory; restart guards</i>
Control-plane separation	<i>STRICT reproducibility locks independent from safety supervision</i>
Memory & retrieval	<i>Deterministic embedding geometry; graph-augmented retrieval; portable memory assembly programs; demand-loaded pyramid</i>
Interconnected instances	<i>Multi-instance coherence; single-turn authority convergence</i>
Learning & evolution	<i>Routing reliability; task-map drift; graph co-activation; artifact-based cloud training loops</i>
Edge deployment	<i>Relevance-scoped VRAM/resident set; small-substrate operation; local presence</i>
Neural articulation	<i>GGUF or equivalent sequence model for language output under control</i>

*Layers may be implemented with opt-in feature flags in production, but the **architecture** must admit full-governance, full-audit operation.*

5. Relation to the paradox of fluency

Statistical language models exhibit **associative depth** only in the sense of **token co-occurrence assembly**. Fluent output mimics understanding because language is the observable surface of cognition. DCM accepts this limitation of the neural substrate and **assigns** assembly through the tripartite contract (§1.1): governed steering establishes how to proceed, the **embedded truth substrate** establishes what is factually grounded for this turn through in-process physics, chemistry, math, and related native engines, and the neural layer articulates how to say it—only after that factual ground has been computed and integrated into the model's operating state. See [\[THE_PARADOXICAL_SHIFT_GGUF_AND_GOVERNED_ASSEMBLY.md\]](#) ([THE_PARADOXICAL_SHIFT_GGUF_AND_GOVERNED_ASSEMBLY.md](#)).

6. Short-form definitions

One sentence:

A DCM is an integrated cognitive model—neural articulation, governed steering, and **embedded factual cognition** (physics, chemistry, math, and related native domains)—in which the checkpoint speaks only after in-process truth has established the factual ground the answer must stand on.

One paragraph (abstract):

Deterministic Cognitive Models (DCMs) constitute an AGI-capable architecture class and the **evolutionary successor** to token-first LLM product design. A DCM is **not** a cloud API router. It is a **single integrated cognitive system** in which physics, chemistry, mathematics, units, logic, and related domains are **built into the architecture** as native factual faculties—not bolt-on plugins or external services. Intelligence converges in three parts: the neural substrate supplies language; the governed runtime supplies intent, maps, procedures, inference control, and supervision; the **embedded truth substrate** activates the relevant in-process domain engines, computes grounded facts, and integrates them into the model's operating state **before** assembly—not after fluency. Cross-domain behavior is replay-stable, evidence-bound, mathematically safe, subconsciously administered, edge-native, and **fully auditable**. Neural checkpoints are interchangeable articulators on small hardware because factual grounding and governance live in the integrated runtime, not in parameter count alone.

Partner abstract (~90 words):

A DCM is not a language model that calls APIs when it gets stuck. **Physics, chemistry, math, and related truth are built into the model itself**—native faculties of the architecture, running in-process on the device. On every turn the system activates the relevant embedded domains, establishes a factual ground under map authority, and only then lets the GGUF articulate language back through that ground. Words from the checkpoint; steering from the governed runtime; **facts from integrated truth**. That is what makes DCM revolutionary—not cloud routing, but a cognitive model with a built-in factual foundation. CRM-FFE is the reference stack.

7. CRM-FFE as reference stack

CRM-FFE (Crowell Reasoning Model, FFE integration fork) is the **reference implementation** of DCM described in this repository: forty-seven registered atomic cognitive formulas, twenty-five system-method protocols, sixteen composition inventions, production validation gates, and operator documentation for rebuild-from-scratch verification. CRM-FFE is a **instance** of DCM, not a synonym for the class. Other conforming implementations may use different map corpora, tool inventories, or neural substrates while satisfying the properties in §3.

8. Suggested citation form

Deterministic Cognitive Model (DCM): an AGI-capable, governed, auditable integrated cognitive architecture—and evolutionary successor to token-first LLM design—in which physics, chemistry, mathematics, and related domains are embedded as native in-process factual faculties (not external API routing), converging with governed steering and controlled neural articulation to produce factually grounded response (CRM-FFE reference implementation).

Canonical definition v1.3 (2026-06-05). Partner- and publication-safe; no proprietary implementation parameters.